

Deep Research: Avatrada 57 Topic 015

Engineering Report: Partial Fill Reconciliation Logic

TO: System Architect, Algorithmic Trading Division **FROM:** Autonomous Technical Researcher **DATE:** October 26, 2023 **SUBJECT:** Deep-Dive on Partial Fill Reconciliation and Stop Loss Management

This report provides a detailed engineering analysis of the Partial Fill Reconciliation system component as specified in the source documentation. The analysis deconstructs the required logic, proposes a robust implementation strategy, and performs a critical analysis of potential failure modes, edge cases, and optimizations.

1. Technical Deconstruction

The core objective is to create an autonomous decision-making framework for managing partially filled orders. This prevents unwanted exposure from stale limit orders and ensures that protective stops are always correctly sized relative to the actual position. The system can be broken down into four key components: Core Parameters, Decision Logic, State Management, and Stop Loss Synchronization.

1.1. Core Parameters

The algorithm's behavior is governed by a set of configurable parameters. These must be defined and calibrated for the specific asset and market conditions.

- **FillRatio**: The ratio of the currently filled quantity to the original order quantity. $\text{FillRatio} = \text{FilledQuantity} / \text{OriginalQuantity}$

- **StallCondition** : A boolean state determined by a combination of time and market activity. An order is considered "stalled" if no new fills have occurred for a duration `T_stall` (e.g., 15 seconds) while the market is still active (i.e., there is trading volume).
- **PriceDeviationThreshold** : A measure of how far the current market price has moved away from the order's limit price. To be robust against varying volatility, this should not be a fixed price amount but rather a dynamic value based on a volatility metric like the Average True Range (ATR).

$$\text{PriceDeviation} = |\text{CurrentMarketPrice} - \text{OrderLimitPrice}|$$

$$\text{PriceDeviationThreshold} = C * \text{ATR}(n)$$
Where `C` is a constant multiplier (e.g., 0.5) and `ATR(n)` is the n-period ATR. An adverse price move is one that makes the order less likely to be filled (e.g., price moving up for a buy limit order).

1.2. Decision Logic Flow

The central task is to decide between "Cancel Remainder" and "Leave Working". The logic, triggered on every partial fill update or on a periodic check, follows these rules:

1. High Fill Ratio Check:

- **IF** `FillRatio > 0.90` (90%): The order is nearly complete. The cost of re-submitting outweighs the risk of the remainder not filling.
- **ACTION**: Leave the order working. The goal is to achieve a full fill.

2. Low Fill Ratio & Adverse Move Check:

- **IF** `FillRatio < 0.50` (50%) **AND** `StallCondition == True` **AND** `PriceDeviation > PriceDeviationThreshold` : This is the critical scenario. The order is significantly incomplete, has stopped filling, and the market is moving away, making a future fill at this price unlikely.
- **ACTION**: Cancel the remaining quantity of the parent order. Immediately trigger Stop Loss Synchronization.

3. Default/Intermediate State:

- **IF** conditions for (1) or (2) are not met: The order is in a transitional state. It may still fill.
- **ACTION**: Leave the order working and continue monitoring.

1.3. State Management

The system must track the state of each parent order and its associated child orders (e.g., Stop Loss). An event-driven architecture is ideal. The state transitions for the parent order would be:

- SENT -> WORKING
- WORKING -> PARTIALLY_FILLED (on first fill)
- PARTIALLY_FILLED -> FILLED (on final fill)
- PARTIALLY_FILLED -> CANCELLED (if reconciliation logic cancels the remainder)
- WORKING -> CANCELLED

The reconciliation logic is primarily active in the PARTIALLY_FILLED state.

1.4. Stop Loss Synchronization

This is a critical risk management function. An incorrect stop quantity can lead to being over-hedged (stop is larger than position) or under-hedged.

- **Mechanism**: The system must maintain a link between the parent entry order and its child stop loss order.
- **Trigger**: This logic is triggered immediately after the parent order's remainder is successfully cancelled.
- **Formula**: The update is a direct mapping of the filled quantity.
$$\text{NewStopLossQuantity} = \text{FinalFilledQuantity}$$
- **Atomicity**: The "Cancel Remainder" and "Update Stop" actions should be treated as a single logical transaction. A failure in the stop update after a successful cancel must trigger a high-priority alert.

2. Implementation Strategy

This section outlines a practical approach to building the described system using common algorithmic trading patterns and libraries.

2.1. Architectural Pattern: Event-Driven

An event-driven architecture is the natural fit. The trading engine would produce events such as `ORDER_FILLED`, `ORDER_STATUS_CHANGED`, etc. Our reconciliation logic would be a "listener" or "handler" that subscribes to these events.

```
# Conceptual Event Handler
def on_order_update(event):
    order = event.order

    if order.status == 'PARTIALLY_FILLED':
        handle_partial_fill(order)

def handle_partial_fill(order):
    # 1. Calculate parameters
    fill_ratio = order.filled_qty / order.original_qty
    is_stalled = check_stall_condition(order)
    price_deviation, threshold = calculate_price_deviation(order)

    # 2. Apply decision logic
    if fill_ratio < 0.5 and is_stalled and price_deviation > threshold:
        # 3. Execute actions
        api.cancel_order(order.id)
        # The subsequent 'CANCELLED' event will trigger stop adjustment

def on_order_cancelled(event):
    order = event.order
    if order.was_partially_filled:
        adjust_stop_loss(order)

def adjust_stop_loss(parent_order):
    stop_order_id = get_linked_stop_id(parent_order.id)
    new_qty = parent_order.filled_qty
```

```

if new_qty > 0:
    api.modify_order(stop_order_id, new_quantity=new_qty)
else: # Should not happen if partially filled, but for safety
    api.cancel_order(stop_order_id)

```

2.2. Pseudocode Algorithm

Here is a more detailed algorithm for the `handle_partial_fill` decision process.

```

FUNCTION handle_partial_fill(order):
    // CONFIGURATION
    HIGH_FILL_THRESHOLD = 0.90
    LOW_FILL_THRESHOLD = 0.50
    STALL_TIMEOUT_SECONDS = 15
    ATR_PERIOD = 14
    ATR_MULTIPLIER = 0.5

    // 1. GATHER DATA
    fill_ratio = order.filled_qty / order.original_qty
    time_since_last_fill = current_time() - order.last_fill_timestamp
    market_price = get_current_market_price(order.symbol)

    // 2. CHECK FOR HIGH FILL
    IF fill_ratio >= HIGH_FILL_THRESHOLD:
        LOG "Fill ratio > 90%. Leaving order working."
        RETURN

    // 3. CHECK FOR LOW FILL & STALLED CONDITION
    is_stalled = (time_since_last_fill > STALL_TIMEOUT_SECONDS)

    IF fill_ratio < LOW_FILL_THRESHOLD AND is_stalled:
        // 4. CHECK FOR ADVERSE PRICE MOVEMENT
        atr_value = get_atr(order.symbol, ATR_PERIOD)
        price_deviation_threshold = ATR_MULTIPLIER * atr_value

        price_deviation = 0
        IF order.side == 'BUY':
            price_deviation = market_price - order.limit_price // Price moved up
        ELSE IF order.side == 'SELL':

```

```

        price_deviation = order.limit_price - market_price // Price moved
down
    IF price_deviation > price_deviation_threshold:
        LOG "Order stalled with adverse price move. Cancelling remainder."
        // Execute cancel. The stop adjustment is handled by the event
listener
        // for the resulting cancellation confirmation.
        broker_api.cancel_order(order.id)
    ELSE:
        LOG "Order stalled but price has not moved away significantly.
Leaving working."
    ELSE:
        LOG "Order is active and not stalled, or in mid-fill range. Monitoring."
END FUNCTION

```

2.3. Required Libraries & Tools

- **Broker API Wrapper:** A library to interact with the trading venue (e.g., `ib_insync` for Interactive Brokers, `alpaca-trade-api` for Alpaca). This is for sending `cancel_order` and `modify_order` requests.
- **Data Analysis Library:** `pandas` and `numpy` are essential for handling time-series data, calculating metrics like ATR, and managing order state.
- **State Machine Library:** Optional but recommended for complex strategies (e.g., `pytransitions`) to formally manage order states and prevent logical errors.

3. Critical Analysis

A robust system must anticipate and handle failures and edge cases.

3.1. Potential Failure Modes

- **API Latency/Failure:** A `cancel_order` request could fail or be delayed. If the system assumes cancellation but the order remains live, it could get more fills.
 - **Mitigation:** The system must wait for an official `CANCELLED` or `REJECTED` confirmation from the broker before proceeding with the stop loss adjustment. Implement a timeout and alerting mechanism if a confirmation is not received.
- **Race Conditions:** A fill message could arrive immediately after the cancel request is sent but before it is processed by the exchange. This can result in a `CANCEL_REJECTED` message because the order is already fully filled.
 - **Mitigation:** The state machine must gracefully handle `CANCEL_REJECTED` events. If the reason is "Order already filled," the logic should transition the order to `FILLED` and update the stop loss to the full original quantity.
- **Loss of Connectivity:** If the system loses connection to the broker, it cannot manage its orders.
 - **Mitigation:** Implement a reconciliation process on startup. The system should query all open orders and their current states from the broker and re-synchronize its internal state before resuming live decision-making.

3.2. Edge Cases

- **Illiquid Markets:** In markets with low liquidity, the `StallCondition` may be triggered frequently under normal conditions.
 - **Mitigation:** The `T_stall` parameter must be significantly longer for illiquid assets. The logic could be disabled entirely for assets below a certain daily volume threshold.

- **Market Open/Close & High Volatility:** Standard ATR values may not be representative during market open auctions or news-driven events. The logic might cancel a good order prematurely.
 - **Mitigation:** Widen the `PriceDeviationThreshold` (e.g., increase the `C` multiplier) or disable the reconciliation logic for the first/last `N` minutes of the trading session.
- **Minimum Order/Lot Sizes:** After cancellation, the filled quantity might be below the minimum required to place a stop loss order on some exchanges.
 - **Mitigation:** The system should be aware of exchange-specific rules. If the filled quantity is too small, it should either liquidate the position immediately with a market order or flag it for manual intervention.

3.3. Potential Optimizations

- **Dynamic Parameter Tuning:** The static thresholds (`0.90` , `0.50` , `T_stall`) could be made dynamic. For example, in a high-momentum market, the system might be more aggressive about cancelling stale orders by lowering the `LOW_FILL_THRESHOLD` and `T_stall` .
- **"Cancel and Replace" (Chase Logic):** Instead of just cancelling, a more advanced version could "chase" the price. If the `PriceDeviation` is significant, the system could cancel the existing order and immediately submit a new one at a more competitive price (e.g., the current best bid/ask). This converts the passive strategy into an aggressive one.
- **Transaction Cost Analysis:** The decision to cancel could incorporate transaction costs. If commissions are high, it may be more cost-effective to leave an order working for longer, even if it is stalled, rather than cancelling and creating a new order later.